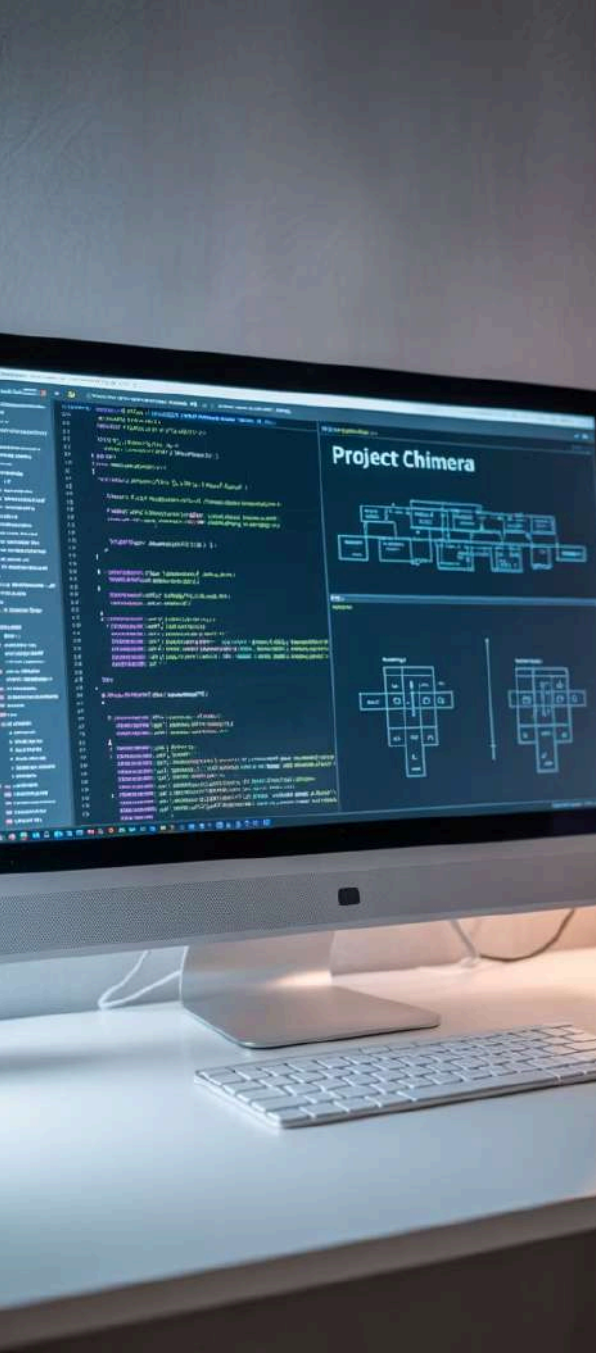




Relacionamentos entre Classes Agregação

UNIVERSIDADE DO OESTE DE SANTA CATARINA - UNOESC

Ciência da Computação | Programação II

Prof.: Leandro Otavio Cordova Vieira



Objetivos da Aula

1 Compreender o conceito de agregação

Definir agregação como um relacionamento especial entre classes onde uma classe contém objetos de outra, mas os objetos podem existir independentemente

2 Diferenciar agregação de associação simples

Identificar as características distintivas que separam agregação de outros tipos de relacionamentos, focando na relação "tem-um" vs "faz parte de"

3 Aplicar agregação em estudos de caso práticos

Implementar exemplos reais usando PHP, demonstrando como modelar agregação em sistemas como biblioteca, escola e e-commerce



Revisão: Associação

Na aula anterior, estudamos **associação** como o relacionamento mais básico entre classes. A associação representa uma conexão onde uma classe conhece e interage com outra classe, mas sem implicar propriedade ou dependência estrutural.

Características da Associação

- Relacionamento "usa-um" ou "conhece-um"
- Classes independentes
- Sem hierarquia de propriedade

Exemplo Prático

Professor *leciona* Disciplina - ambos existem independentemente, mas há uma relação funcional entre eles durante o período letivo

O que é Agregação?

Agregação é um tipo especial de associação que representa um relacionamento "**tem-um**" ou "**parte-todo**" entre classes. Na agregação, uma classe (o todo) contém objetos de outras classes (as partes), mas essas partes podem existir independentemente do todo.

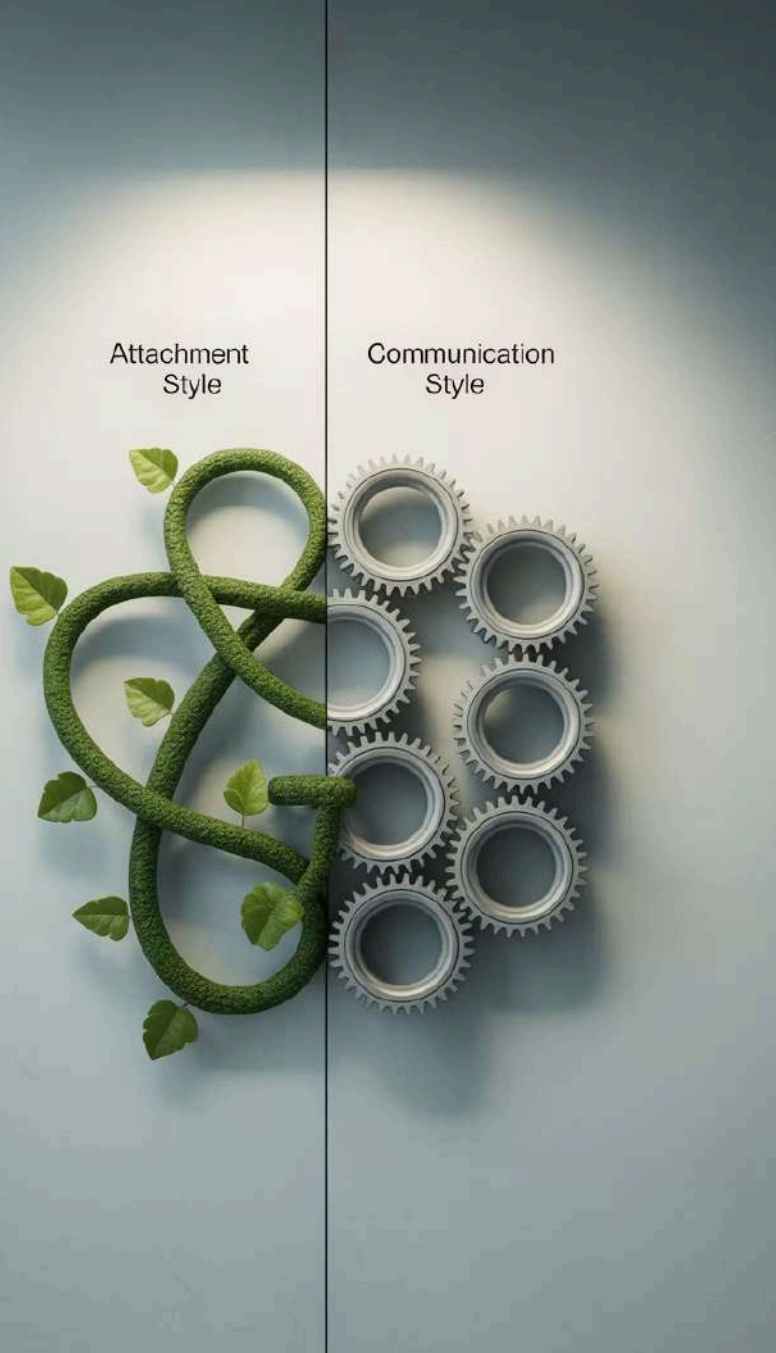


Características Principais

- Relacionamento hierárquico suave
- Partes podem existir sem o todo
- Representa coleções ou agrupamentos
- Ciclo de vida independente dos objetos

Exemplo Conceitual

Uma **Biblioteca** tem muitos **Livros**. Se a biblioteca for fechada, os livros continuam existindo e podem ser transferidos para outra biblioteca



Diferença entre Associação e Agregação

Associação Simples

- **Natureza:** "Usa-um" ou "conhece-um"
- **Dependência:** Funcional apenas
- **Exemplo:** Motorista dirige Carro
- **Característica:** Relacionamento temporário

O motorista pode dirigir diferentes carros, e o carro pode ter diferentes motoristas

Agregação

- **Natureza:** "Tem-um" ou "contém"
- **Dependência:** Estrutural suave
- **Exemplo:** Time tem Jogadores
- **Característica:** Coleção organizada

O time é composto por jogadores, mas jogadores podem existir sem o time e mudar de equipe

Representação em UML

Em diagramas UML, a agregação é representada por um **diamante branco** (losango vazio) na extremidade da linha que conecta as classes. O diamante sempre fica no lado da classe que "contém" ou "agrega".

01

Identificar as classes envolvidas

Determine qual classe é o "todo" e qual é a "parte"

02

Desenhar a linha de conexão

Conecte as duas classes com uma linha reta

03

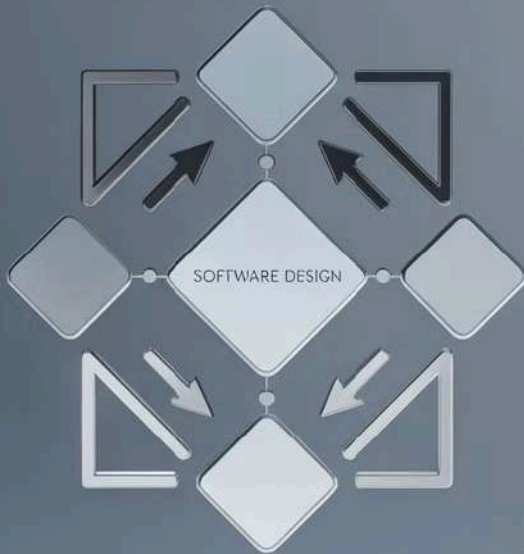
Adicionar o diamante branco

Coloque o diamante vazio no lado da classe agregadora

04

Definir multiplicidade

Especifique quantos objetos participam da relação (1, *, 0..1, etc.)





Exemplo em PHP – Biblioteca

Vamos implementar um exemplo prático onde uma **Biblioteca** agrega vários **Livros**. Este é um caso clássico de agregação onde a biblioteca organiza e gerencia livros, mas os livros têm existência própria.



Classe Livro

Representa livros individuais com propriedades como título, autor e ISBN. Cada livro existe independentemente



Classe Biblioteca

Gerencia uma coleção de livros, permitindo adicionar, remover e listar livros do acervo



Relacionamento

Agregação: Biblioteca "tem" livros, mas livros podem existir em outras bibliotecas

Código em PHP – Biblioteca e Livro

```
class Livro {
    private $titulo;
    private $autor;
    private $isbn;

    public function __construct($titulo, $autor, $isbn) {
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->isbn = $isbn;
    }

    public function getTitulo() {
        return $this->titulo;
    }
}

class Biblioteca {
    private $livros = [];
    private $nome;

    public function __construct($nome) {
        $this->nome = $nome;
    }

    public function adicionarLivro(Livro $livro) {
        $this->livros[] = $livro;
    }

    public function listarLivros() {
        foreach($this->livros as $livro) {
            echo $livro->getTitulo() . "\n";
        }
    }
}
```

Note como a Biblioteca **contém** objetos Livro, mas não os cria nem os destrói – característica típica da agregação

Exemplo em PHP – Turma e Aluno

```
class Aluno {
    private $nome;
    private $matricula;

    public function __construct($nome, $matricula) {
        $this->nome = $nome;
        $this->matricula = $matricula;
    }

    public function getNome() {
        return $this->nome;
    }
}

class Turma {
    private $alunos = [];
    private $disciplina;
    private $periodo;

    public function __construct($disciplina, $periodo) {
        $this->disciplina = $disciplina;
        $this->periodo = $periodo;
    }

    public function matricularAluno(Aluno $aluno) {
        $this->alunos[] = $aluno;
    }

    public function listarAlunos() {
        foreach($this->alunos as $aluno) {
            echo $aluno->getNome() . "\n";
        }
    }
}
```

Alunos podem existir sem uma turma específica e podem estar em múltiplas turmas simultaneamente





Exemplo em PHP – Pedido e Produto

```
class Produto {
    private $nome;
    private $preco;
    private $codigo;

    public function __construct($nome, $preco, $codigo) {
        $this->nome = $nome;
        $this->preco = $preco;
        $this->codigo = $codigo;
    }

    public function getPreco() {
        return $this->preco;
    }
}

class Pedido {
    private $produtos = [];
    private $numeroPedido;
    private $dataCreacao;

    public function __construct($numeroPedido) {
        $this->numeroPedido = $numeroPedido;
        $this->dataCreacao = date('Y-m-d H:i:s');
    }

    public function adicionarProduto(Produto $produto) {
        $this->produtos[] = $produto;
    }

    public function calcularTotal() {
        $total = 0;
        foreach($this->produtos as $produto) {
            $total += $produto->getPreco();
        }
        return $total;
    }
}
```

Produtos existem independentemente de pedidos e podem estar em vários pedidos diferentes



Estudo de Caso: Biblioteca

Analisando o sistema de biblioteca completo, identificamos múltiplas agregações que modelam eficientemente as relações do mundo real



Biblioteca ◇ Livro

Uma biblioteca agrega muitos livros (1.*).
Livros podem existir sem biblioteca

Biblioteca ◇ Funcionário

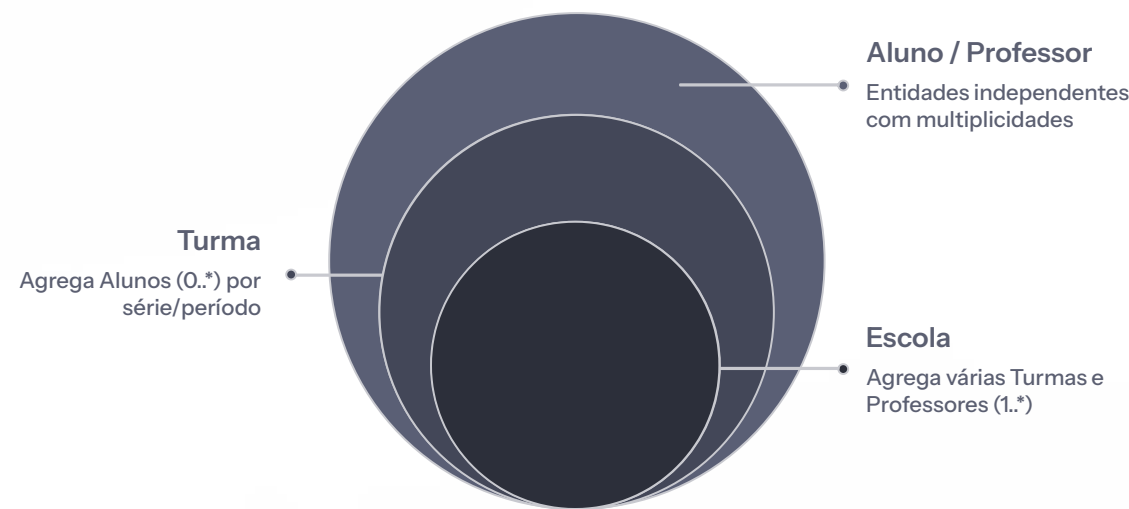
Biblioteca tem funcionários (1.*).
Funcionários podem trabalhar em outras bibliotecas

Livro ◇ Autor

Livro pode ter múltiplos autores (1.*).
Autores existem independentemente

Estudo de Caso: Escola

O sistema escolar apresenta vários exemplos de agregação, onde entidades organizacionais contêm outras entidades que mantêm sua independência

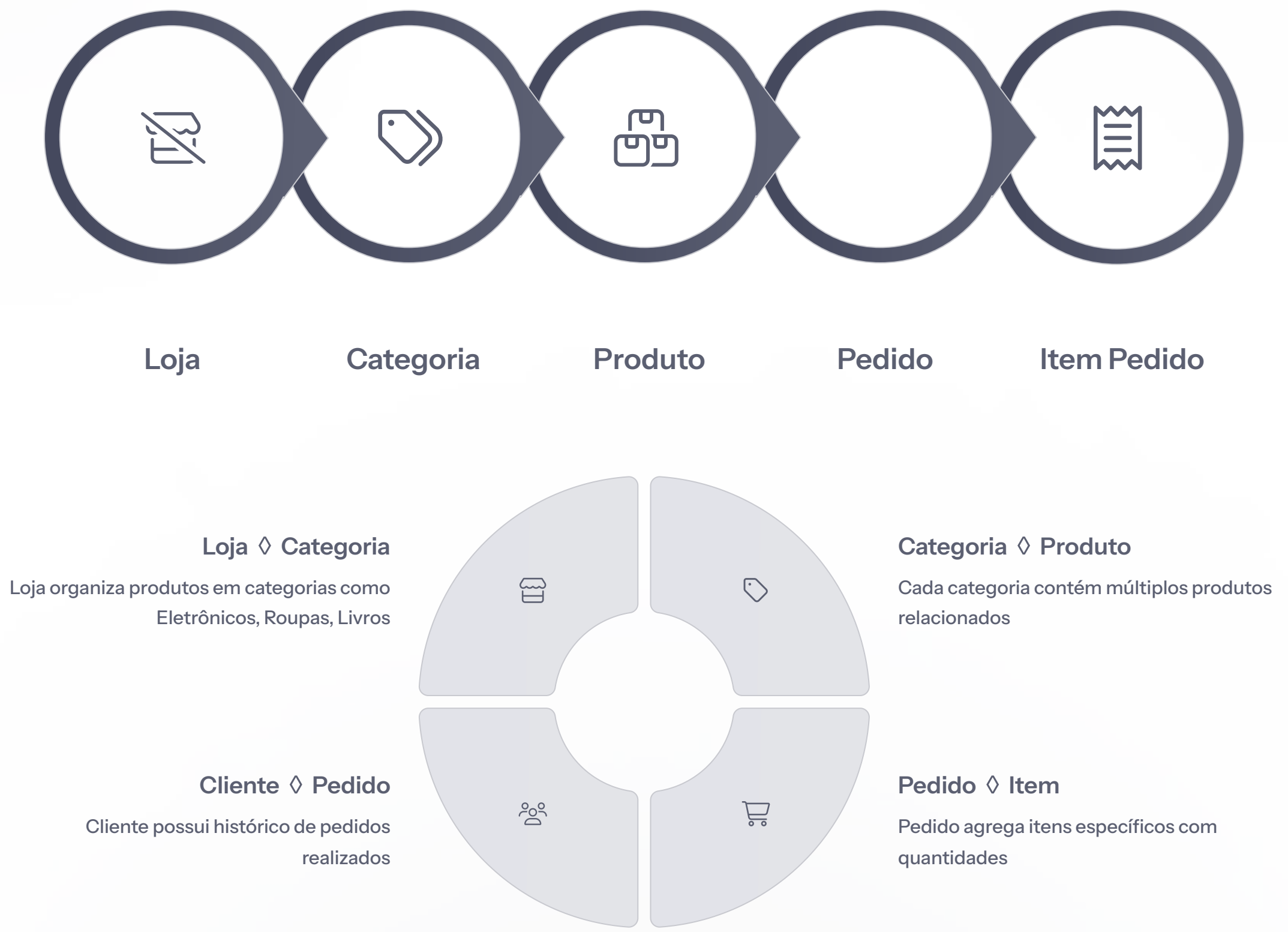


- 1 Escola ◇ Turma
Escola contém várias turmas organizadas por série e período
- 2 Turma ◇ Aluno
Turma agrega alunos matriculados na disciplina específica
- 3 Escola ◇ Professor
Escola tem professores que podem lecionar em múltiplas turmas



Estudo de Caso: E-commerce

Sistemas de e-commerce são ricos em relacionamentos de agregação, especialmente na gestão de pedidos, produtos e categorias





Benefícios da Agregação

A agregação traz vantagens significativas para o design de software, promovendo melhores práticas de programação orientada a objetos



Organização Clara

Estrutura hierárquica natural que reflete relacionamentos do mundo real, facilitando a compreensão do sistema pelos desenvolvedores e stakeholders



Clareza Conceitual

Expressa claramente relações "tem-um" e "parte-todo", tornando o código mais intuitivo e auto-documentado



Modularidade Aprimorada

Permite reutilização de componentes em diferentes contextos, já que as partes mantêm independência do todo



Flexibilidade de Design

Facilita mudanças e extensões no sistema sem quebrar relacionamentos existentes

Erros Comuns

Desenvolvedores frequentemente confundem agregação com outros tipos de relacionamento, especialmente composição. Compreender essas diferenças é crucial para um design efetivo

Confundir Agregação com Composição

Erro: Usar agregação quando há dependência vital entre objetos

Correto: Agregação = independência; Composição = dependência total

Exemplo: Casa e Quarto é composição (quarto não existe sem casa)

Usar Agregação Desnecessariamente

Erro: Aplicar agregação em associações simples

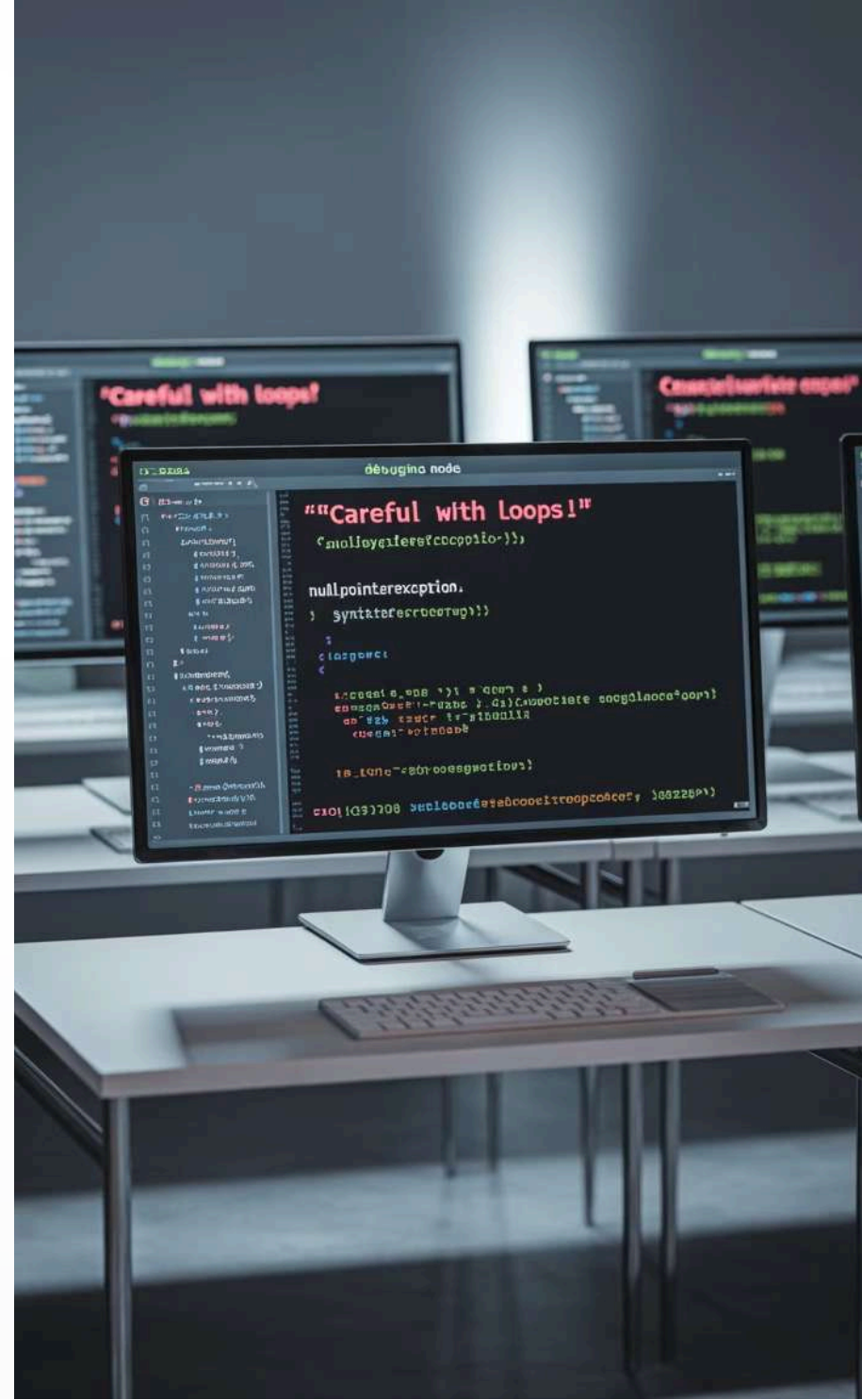
Correto: Agregação implica coleção ou agrupamento lógico

Exemplo: Professor leciona Disciplina é associação, não agregação

Ignorar Ciclo de Vida dos Objetos

Erro: Não considerar se objetos podem existir independentemente

Correto: Na agregação, partes sobrevivem sem o todo



Atividade em Grupo

Mini-Sistema com Agregação

Trabalhem em grupos de 3-4 pessoas para criar um mini-sistema que demonstre o uso prático da agregação

01

Escolher um Domínio

Selecione um área como: Hospital, Universidade, Empresa, Loja, ou Sistema Bancário

02

Identificar 4 Classes

Definam ao menos 4 classes relacionadas com pelo menos 2 agregações claras

03

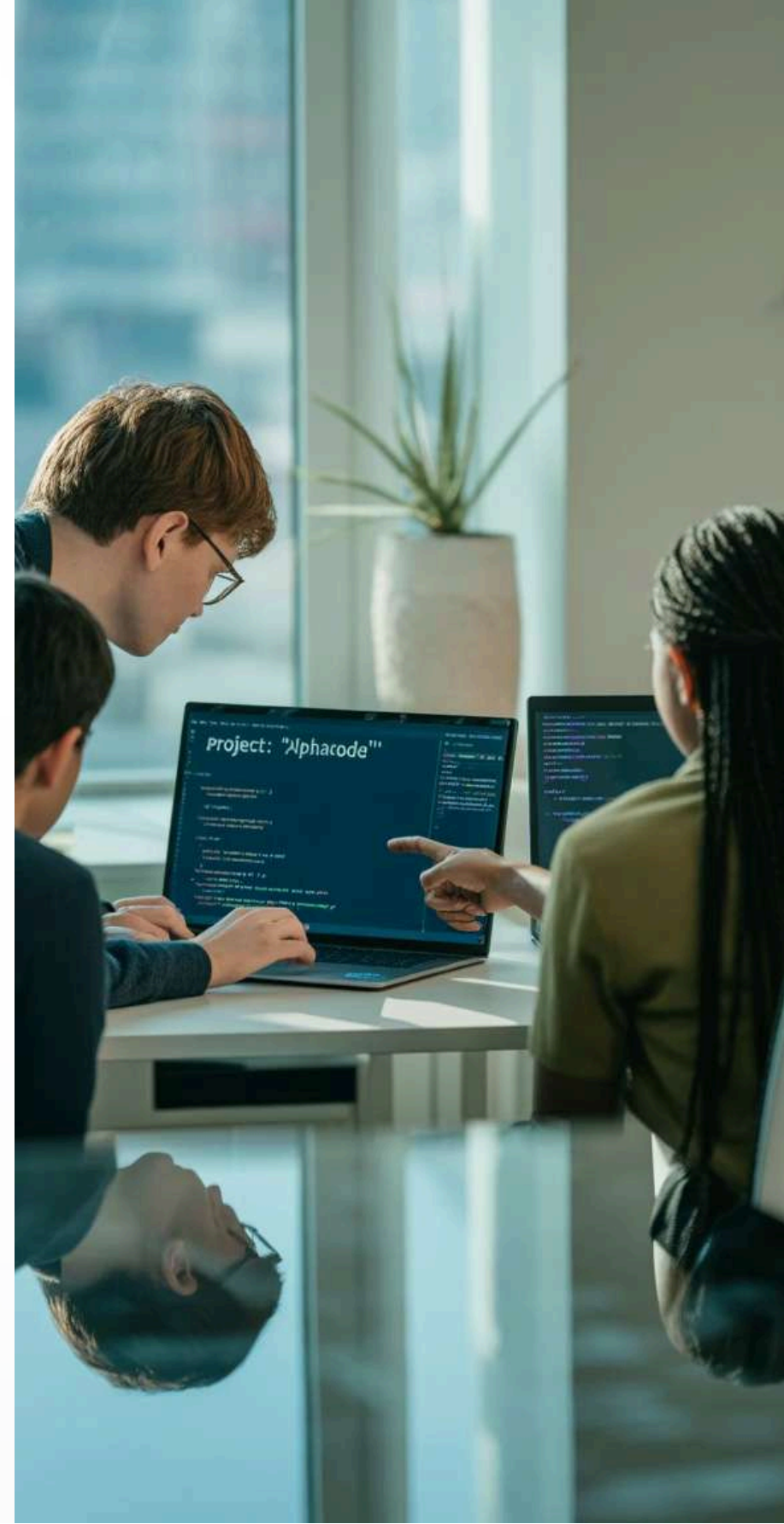
Implementar em PHP

Criem o código PHP com construtores, métodos e relacionamentos de agregação

04

Criar Diagrama UML

Desenhem o diagrama UML mostrando as agregações com diamantes brancos



Orientações da Atividade

Para uma entrega completa e bem fundamentada, sigam estas orientações específicas durante o desenvolvimento da atividade

Justificativa de Modelagem

Expliquem **por que** escolheram agregação em vez de associação simples ou composição para cada relacionamento

- Demonstrem independência das partes
- Mostrem a relação "tem-um"
- Evidenciem o agrupamento lógico

Documentação Técnica

Incluam comentários no código explicando:

- Propósito de cada classe
- Relacionamentos entre classes
- Métodos principais e sua função

Exemplo de Uso

Criem um script que demonstre:

- Criação de objetos
- Estabelecimento da agregação
- Uso dos métodos implementados



Discussão em Grupo

Momento de compartilhamento e aprendizado coletivo onde cada grupo apresentará sua solução e receberá feedback construtivo

1 — Apresentação (5 min)

Cada grupo apresenta brevemente seu sistema, classes escolhidas e justificativas de design

2 — Análise Crítica (3 min)

Outros grupos fazem perguntas e sugerem melhorias ou alternativas de modelagem

3 — Feedback do Professor (2 min)

Avaliação técnica focada nos conceitos de agregação e boas práticas aplicadas

📌 **Dica:** Prestem atenção nas diferentes abordagens dos colegas - várias soluções podem estar corretas para o mesmo problema!



Resumo da Aula

Consolidamos hoje conhecimentos fundamentais sobre agregação, estabelecendo bases sólidas para relacionamentos mais complexos em orientação a objetos

Conceito de Agregação

Relacionamento "tem-um" onde o todo contém partes que podem existir independentemente. Representado por diamante branco em UML

Diferença da Associação

Agregação implica organização e agrupamento lógico, enquanto associação simples representa apenas conhecimento ou uso entre classes

Exemplos Práticos

Biblioteca-Livro, Turma-Aluno, Pedido-Produto demonstraram aplicações reais da agregação em sistemas comerciais e educacionais

A agregação é uma ferramenta poderosa para modelar relacionamentos naturais do mundo real, promovendo código mais organizado, reutilizável e intuitivo

Project: Aurora

- ✓ Corpebd
- ✓ Secure funding
- ✓ U tonedect
- ✓ Finalize design
- ✓ Finalize design
- ✓ Finalize design
- ✓ Coneets orlcd.
- ✓ Secur chy ouued

Próxima Aula

Composição

Na próxima aula, avançaremos para o relacionamento mais forte entre classes: a **composição**. Veremos como ela difere da agregação e quando aplicar cada tipo de relacionamento



Agregação

Diamante branco
Independência das partes



Composição

Diamante preenchido
Dependência vital das partes

Preparem-se para entender como modelar relacionamentos onde as partes não podem existir sem o todo, como Carro e Motor, ou Casa e Cômodos